A Message Cannot Arrive Before It Is Sent

Physical-Consistency Auditing for Streaming Latency Benchmarks, and What It Left of a Kafka-versus-Redis Comparison

GUSTAVO PEDRO RICOU, School of Computer Science and Statistics, Trinity College Dublin, Ireland

We set out to answer an ordinary question: for a real-time sports data feed, does the choice between Apache Kafka and Redis Streams affect end-to-end delay, and how does that change with the number of concurrent feeds? We built the benchmark, drove it with 3,315 real football matches at their true event rate, and obtained a clean answer — Redis broker delay rising monotonically with concurrency while Kafka stayed flat, $p = 9.0 \times 10^{-11}$, no overlap between the two systems' run distributions, exactly as a single-threaded server should behave.

It was an artefact. Broker delay subtracts a timestamp taken in the producer process from one taken in the consumer process, so it admits a check no statistic supplies: the result cannot be negative. Applying that check to every run, rather than to the runs whose results looked wrong, rejected 1,321 of 2,266 runs, including every run behind the finding above. The rejected data is invisible to conventional inspection: medians stay positive, intervals stay narrow, effect sizes stay large, and the direction agrees with theory.

We report the check as a stated rule with its sensitivity analysis, and re-run the study inside it. Three results survive and none is the one we set out to find. The two brokers are statistically equivalent within 1 ms and neither degrades across the concurrency range, a conclusion we show is robust to the unequal retention the check itself introduces. The end-to-end difference between the systems lives entirely in client code rather than in either broker, and exists *only* at the sparse arrival rate of the real workload: replay the same data ten times faster and it disappears, which is the regime every synthetic-publisher benchmark measures. And each system has exactly one client setting worth one to two orders of magnitude in delay, both of which are free on a co-located testbed and therefore invisible to the way such settings are normally evaluated.

CCS Concepts: • **Computer systems organization → Distributed architectures**; • **Software and its engineering → Software performance**.

Additional Key Words and Phrases: streaming systems, latency benchmarking, measurement validity, Apache Kafka, Redis Streams, reproducibility

1 Introduction

Message brokers carry event data between producers and consumers in most real-time analytics pipelines, and two products dominate the role. Apache Kafka [11] stores messages in a partitioned, replicated log written to disk, designed for durability and high throughput. Redis Streams [21] keeps them in memory in an append-only structure, designed for minimum delay per operation. Practitioners choose between them largely on grey-literature comparisons that report Redis as several times faster [3, 9, 28], none of which is reproducible and none of which uses a workload from the domain the reader works in.

1.1 The original question

This work began as a straightforward attempt to fix that, for one domain:

> *Using the publicly available StatsBomb open dataset (2003–2023) and open-source load-generation scripts, compare end-to-end lag between Redis Streams and Apache Kafka for real-time sports data feeds, under varying concurrency.*

The domain supplied something a synthetic benchmark cannot: a real arrival process, and a real answer to *how many feeds run at once*. Football match records carry kick-off times, so the number of

Author's Contact Information: Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland, pedrorig@tcd.ie.

simultaneous feeds is an empirical quantity rather than a swept parameter. We treat that workload throughout as the *setting* that produced our findings rather than as a contribution in itself, and Section 3 describes it only to the depth needed to interpret the results.

1.2 The answer we obtained first

Our first campaign returned a clean result. Replaying distinct real matches across one, five and ten concurrent feeds, Redis broker delay rose monotonically (1.006 → 1.197 → 1.346 ms) while Kafka stayed flat to within 0.3%. Every comparison was significant after Holm correction [7], the strongest at $p = 9.0 \times 10^{-11}$, and at five feeds and above the two systems' run distributions did not overlap at all.

The finding was not merely significant; it was explicable. A Redis server executes commands on a single thread, so concurrent streams contend for one execution context, whereas Kafka's partitioning admits genuine parallelism. The measurement appeared to reproduce the textbook architectural distinction between the two designs, in the predicted direction and at the predicted scale.

It was an artefact. Broker delay is computed as consumer receipt minus broker acknowledgement, two timestamps taken in two processes. A message cannot arrive before it is sent, so a negative value is not measurement noise but proof that the instrument has failed. Checking every run against that constraint — rather than only the runs whose results looked implausible — rejected 862 of 1,382 single-machine runs and 459 of 884 multi-machine runs, including every run behind the result above.

1.3 Why the failure is worth a paper

Timestamp inversion under load is invisible to the checks a reviewer would apply. Medians stay positive and plausible. Confidence intervals stay narrow. Effect sizes are large and the direction agrees with theory. Only 5 to 14% of individual events invert, which is enough to displace a median by a fraction of a millisecond in a measurement whose entire effect is a fraction of a millisecond, and far too little to make any aggregate look wrong.

A benchmark can therefore produce a complete, internally consistent and entirely false result set. No amount of statistical care exposes it, because the contamination is systematic rather than random: greater care produces tighter intervals around the wrong number. Only a check against a physical constraint exposes it, and it must be applied to every condition as a rule.

1.4 Contributions

(1) **A physical-consistency audit for cross-process latency benchmarks** (Section 4.3), stated as a rule, applied uniformly to all 2,266 runs, with its threshold sensitivity reported rather than asserted (Section 6). We show the audit binds hardest exactly where the measured effect is smallest, which inverts the usual intuition that large clean effects are the trustworthy ones.

(2) **A demonstration**, which we regard as the strongest of the four: a complete, significant, theory-confirming result set that was physically impossible, reported in full so a reader can judge how convincing invalid data looks (Section 5).

(3) **An arrival-rate result** (Section 7.2): the end-to-end difference between the two systems exists only at the real workload's sparse rate and vanishes under accelerated replay. Since every benchmark in Section 2 drives its system with a dense synthetic publisher, this is a regime in which the standard method reports parity by construction.

(4) **A configuration result** (Section 7.4): each system has exactly one client-side setting worth one to two orders of magnitude in delay, and both are free on a co-located testbed.

2 Background and Related Work

2.1 Streaming benchmarks

Stream processing has a mature benchmark literature. Linear Road [1] is the canonical application benchmark; NEXMark [26] models an online auction and has become the standard query suite for dataflow engines; ESPBench [5] constructs an enterprise workload; RIoTBench [23] supplies IoT dataflows; and the OpenMessaging Benchmark [19] measures broker throughput and delay directly across messaging systems. Stonebraker et al. [25] set the requirements these evaluate against.

Two properties of this body of work matter here. Each drives its system with a synthetic arrival process chosen for analytical tractability. And none, to our knowledge, reports what fraction of its runs it discarded on integrity grounds. Mohammad [18] surveys benchmark practice in Kafka event-streaming systems and finds reported results frequently not comparable, because client configuration, acknowledgement semantics and instrumentation differ between the systems under test. Our experience supports that critique and extends it: comparable configuration is not sufficient, because the instrument itself can fail silently under load.

2.2 Measuring time across processes

Lamport [14] established that a distributed system has no single physical time, only a partial order induced by causality. Our check is the simplest application of that principle: a broker's acknowledgement causally precedes the consumer's receipt, so a measurement in which receipt is timestamped earlier violates the causal order and supports no conclusion.

The engineering problem is well known and partially solved. Systems approximate physical time with synchronisation protocols, of which NTP [17] is the most widely deployed; Spanner [2] goes further and exposes clock *uncertainty* to the application, on the grounds that a system which pretends to know the time exactly will eventually be wrong undetectably. Closest to our setting, Cloudprofiler [27] addresses precisely the inadequacy we encountered — that millisecond-grade synchronisation is too coarse to profile modern streaming engines — by calibrating time-stamp counters across nodes during quiescent periods, reaching roughly 92 μs accuracy.

We therefore make a narrower claim than "nobody has considered this". The accuracy problem is recognised and better instruments exist. What appears to be missing is the *practice*: an audit applied uniformly to every run of a campaign, with the rejection rate reported the way a survey reports its response rate. Cloudprofiler improves the instrument; we argue that whatever instrument is used, its output should be checked against causality and the check should be published. Our own failure is also not one of clock synchronisation — both processes read the same operating-system clock on one machine — but of *when the timestamp is taken*: a thread descheduled between the event and the call recording it stamps a time later than the event, and under CPU saturation that delay routinely exceeds the interval being measured.

Jain [8] sets out the standard requirement that an instrument be validated on the system it will measure. The check in Section 4.3 is a cheap, specific instance of that for cross-process delay.

2.3 Statistical methods

Latency distributions are non-normal, so difference tests are rank-based: Mann–Whitney U [16] per condition and Kruskal–Wallis [12] across concurrency levels, with Holm's correction [7]. Because several claims here are negative, we state them with two one-sided tests [13, 22] against a pre-specified margin rather than by failing to reject equality. As a point estimate of the difference between two systems we use the Hodges–Lehmann shift [6] with percentile bootstrap intervals [4]; Section 6.4 shows that this estimator's breakdown point is what makes our main conclusion robust to the audit's own side effects.

Table 1. The workload, from 3,315 matches. Medians across matches; peak rate is the maximum over any sliding ten-second window.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

3 The Setting

The workload is a live football event feed, and we describe it only as far as the results require. Full characterisation of 3,315 matches from the StatsBomb open dataset [24], spanning 52 competition-seasons from 2003 to 2023, is available in the artefact; event data of this kind is collected by trained human operators, whose accuracy has been studied [15, 20].

Three properties of the workload drive every experimental choice (Table 1).

It is sparse. A match produces a median of 3,507 events over roughly 8,412 seconds, a mean rate of **0.415** events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes Section 7.2's result possible.

It is bursty. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45. Half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts, which is the duty cycle under which a client that must wake from idle pays that cost on nearly every event.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile, the structural league bound, and the observed maximum. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

4 Method

4.1 Testbeds

Because several of our earlier numbers proved to be properties of a machine rather than of either broker, we state both testbeds and measure the floors that bound resolution.

Testbed A (single machine). Windows 11, 8-core AMD Ryzen, 31 GB RAM, CPython 3.9. Producer and consumer are native processes; Kafka 4.1.1 and Redis 7.2.4 run in Docker containers whose engine executes in a WSL2 virtual machine, reached through published ports. Two floors bound it: a requested 1 ms sleep takes a median of 15.6 ms (the Windows timer tick), and opening a TCP connection to a published port costs 5.6–9.9 ms.

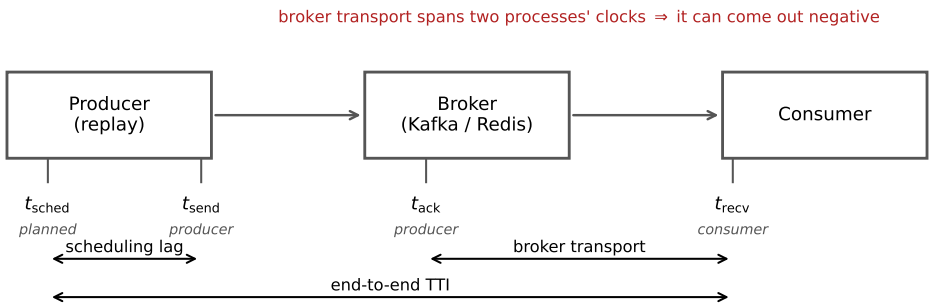

Fig. 1. The four timestamps and the intervals each measure spans. Scheduling lag is measured within one process. Broker transport subtracts a producer-side timestamp from a consumer-side one, and can therefore come out negative when either process is delayed.

Testbed B (four machines). Four Oracle Cloud `VM.Standard.E5.Flex` instances — three brokers and one driver — on Ubuntu 22.04, CPython 3.10, over a private network, with client libraries pinned identically to Testbed A. Both floors disappear: a 1 ms sleep takes 1.06 ms, and a broker round trip on an established connection is 0.22 ms locally and 0.24 ms across machines. A genuine two-machine network is thus *faster* than Testbed A's co-located path, which is the sharpest statement we can offer of how much a testbed distorts a benchmark. Every result reported as a finding comes from Testbed B.

4.2 Measurements

Each match is preprocessed into a replay plan recording, per event, the time at which it should be emitted. A producer replays a plan at a configurable speed-up; a consumer records arrivals. In the concurrency experiment each of the N feeds carries a *distinct* real match at its true rate: an earlier design gave every feed the same merged ten-match plan, which made the concurrency axis covertly a throughput axis.

The primary measure is **Time-to-Insight** (TTI), from an event's scheduled emission to its availability at the consumer. We decompose it (Figure 1):

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{broker transport}} + \text{residual}.$$

Scheduling lag is measured entirely inside the producer and characterises the client. Transport spans the broker and characterises the product. The distinction is not cosmetic: for this workload the two have opposite answers, and reporting only the aggregate is what concealed the result in Section 7.2.

Critically, t_{ack} is stamped in the *producer* process (from Kafka's send callback, or on return from Redis's `XADD`) while t_{recv} is stamped in the *consumer*. That subtraction is the one that can go wrong.

4.3 The consistency check

We implement the causal constraint as a stated rule, fixed before the final campaign and applied to every run including those whose results looked reasonable. A run is **rejected** if either

(1) more than 1% of its events carry a negative value in any latency component, or
(2) the median of any component is negative.

Table 2. **The first result set, later rejected.** Broker transport (median, ms) by concurrency, Testbed A, 10× replay. Every cell fails the check of Section 4.3.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

A condition is usable only if all of its runs survive. Where a condition is partly rejected we report the retention rate, and Section 6.4 bounds the selection this introduces. The tool exits non-zero when any run fails, so a campaign script can stop on it.

4.4 Statistics

Margins are fixed before testing and proportionate to the quantity compared: $\delta = 1$ ms for broker transport, a quantity of order one millisecond, and $\delta = 40$ ms for end-to-end TTI, one video frame at 25 fps. Holm families are declared over the design actually executed: the four per-N transport comparisons form one family, the network-delay arm another; omnibus Kruskal–Wallis tests are uncorrected; equivalence tests are pre-specified and belong to no family. Surviving samples range from 8 to 35 runs per cell; the 8-run cell is treated as descriptive and flagged wherever it appears.

5 The Answer We Obtained First

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

5.1 The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (Table 2). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

5.2 The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

(1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
(2) *A saturating replay speed.* At 120× the producer could not keep pace, inflating scheduling lag to tens of seconds.
(3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median TTI at one feed fell from 1,644 ms to 16 ms once the Kafka producer was pipelined.
(4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design described in Section 4.2.
(5) *A heavy tail contaminating every mean.* Kafka's end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile

Table 3. Consistency audit of every run in the study.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

that ignores load is not system behaviour; decomposition located it in scheduling lag, not transport.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median ≈170 bytes) and serialisation costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgement experiment of Section 7.3 reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid, and nothing in its output said so.

6 The Audit

6.1 What it rejects

Applying the rule to all 2,266 runs rejects 1,321 of them (Table 3). On Testbed A only 8 of 76 conditions survive intact; on Testbed B, 13 of 40. No condition behind Table 2 survives.

6.2 Why it fails, and why it looked correct

The cause is legible in *which* conditions fail: every rejected condition in the concurrency corpus was replayed at ten times real speed, and every condition at true real time passes. The mechanism is the one described in Section 2.2. The acknowledgement timestamp is taken by the producer's callback thread; when accelerated replay saturates the driver's CPU that thread is descheduled, and by the time it reads the clock the consumer has already recorded receipt. It is a timestamping race under load, not clock skew, which is why medians stay positive at moderate load and invert wholesale only at saturation. At one hundred connections the failure becomes gross enough to see unaided: Kafka's median transport is reported as −6.4 ms.

The corpus looked healthy for an arithmetic reason. Between 5 and 14% of events invert, which displaces a median by a few tenths of a millisecond in a measurement whose entire effect is 0.34 ms. The bias is also *asymmetric* between arms, because the two clients record the acknowledgement differently — Kafka from an asynchronous callback, Redis on return from a blocking command — and an asymmetric bias between two arms of a comparison manufactures a difference where none exists.

6.3 How much the threshold matters

We expected the distribution of inversion rates to be clearly bimodal, which would have made the 1% threshold uncontroversial. It is not (Figure 2a): only 104 of 1,382 runs are completely clean and

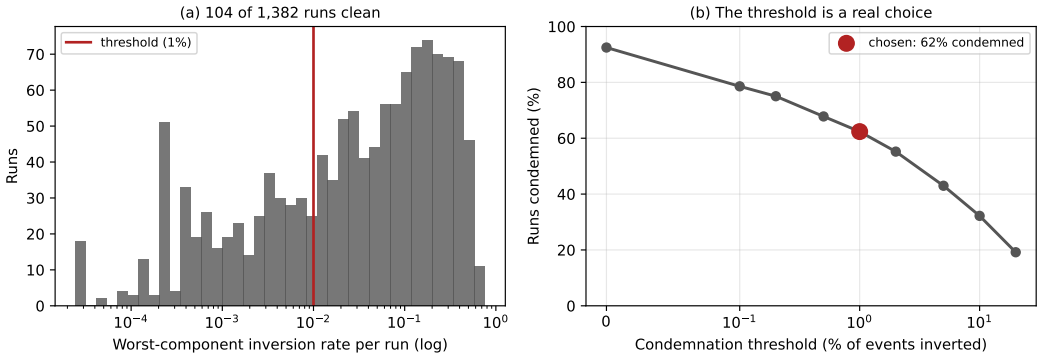

Fig. 2. The audit on Testbed A. (a) Per-run inversion rates: 104 runs are clean and the rest spread over three decades, so there is no natural threshold. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

the rest spread over three orders of magnitude with no natural gap. The threshold is a real decision, so we report its consequences rather than asserting robustness.

It strongly affects *how many runs* are rejected — from 92.5% at zero tolerance to 19.2% at a permissive 20% (Figure 2b). It does not affect *which conditions are usable*, and therefore changes no conclusion here: a condition requires all its runs to survive, and even at a 5% threshold the accelerated arms retain only 32–77% of theirs. The withdrawal of Table 2 holds across the entire range.

6.4 Bounding the audit's own selection effect

An audit that rejects unequally between arms introduces selection. In our concurrency experiment Kafka passes on 100% of runs at every level while Redis passes on 53–100%. Since inversion is caused by driver saturation, the surviving Redis runs are plausibly the less-loaded ones, and the bias would run in exactly the direction that manufactures our reported equivalence.

The rejected runs' values are not recoverable, so a threshold sweep is impossible. Instead we bound: impute every rejected Redis run at a hypothetical value and ask how large it must be before the conclusion changes (Table 4). Equivalence survives at every level, and no imputed value up to 1 second breaks it.

The reason is structural, and so is its limit. The Hodges–Lehmann shift is a median of pairwise differences and has a breakdown point of one half: while more than half the runs survive, nothing the rejected runs could have contained moves the estimate outside the margin. Our tightest cell is N=12 at 52.8% retention — only 2.8 points above breakdown. Had retention fallen below one half this defence would not apply, and we would have had to withdraw the cell.

6.5 The property that makes this general

One feature transfers beyond this study. The check tests against zero, so it rejects a measurement in proportion to how close that measurement sits to zero. Our network-delay arm, where Redis transport is tens of *seconds*, passes at 15 runs out of 15 at every injected delay — on the same hardware, in the same campaign, minutes apart from conditions that fail outright. A few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one.

The check therefore bites hardest exactly where the scientific question is most delicate. This inverts a common intuition. In cross-process delay measurement, an effect of the same order as the

Table 4. Bounding the unequal-retention selection effect. “Worst case” imputes every rejected Redis run at the largest value observed in that cell. Margin 1 ms.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	+0.081	+0.081	yes
9	20/27	+0.021	−0.051	yes
10	17/30	+0.116	−0.485	yes
12	19/36	+0.053	−0.227	yes

Table 5. End-to-end delay and its decomposition, Testbed B, true real-time replay, after the consistency check (164 of 201 runs retained). Medians, ms.

	TTI		Sched. lag		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

instrument’s own noise floor is the one most likely to have been manufactured by it, and statistical significance offers no protection because the artefact is systematic. We recommend that streaming benchmarks report an integrity audit as a matter of course.

6.6 What we withdraw

We withdraw the entire Testbed A arm, and on Testbed B the accelerated concurrency sweep, the connection sweep above ten connections, and the three-node cluster arm, which fails on every run in both systems.

7 What Survives

All results below come from Testbed B and pass the check. Each states its retention.

7.1 The brokers are equivalent and neither degrades

At the concurrency levels the workload produces, with each feed carrying a distinct real match at true real time, 164 of 201 runs survive. Broker transport is flat across the range: Kruskal– Wallis gives $p = 0.061$ for Kafka and $p = 0.091$ for Redis, neither significant, with a largest-to-smallest median ratio of 1.06 and 1.17 (Table 5). At $N \in \{9, 10, 12\}$ all three procedures agree that transport is equivalent within 1 ms, with Hodges–Lehmann shifts of 0.021, 0.116 and 0.053 ms; Section 6.4 shows this survives the audit’s selection effect. At N=1 the procedures disagree, because one Kafka run carries a 6.3 ms start-up outlier that pulls the mean; with eight runs per system we report the disagreement rather than resolving it.

This directly contradicts Table 2, and the contradiction is the audit’s doing.

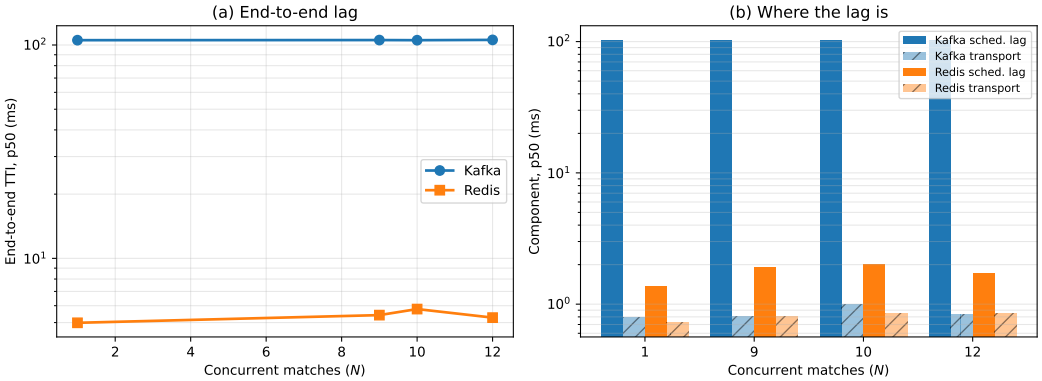

Fig. 3. (a) End-to-end delay against concurrency, Testbed B, after the check. (b) The same runs decomposed: the twentyfold difference is entirely producer scheduling lag, while the brokers' own transport is separated by well under a millisecond. Log scale.

7.2 The end-to-end difference is not in the broker, and exists only at the real rate

TTI and transport give opposite answers, and the decomposition explains why (Figure 3). Of Kafka's 105.5 ms, 102.9 **ms is producer scheduling lag**, against Redis's 1.8 ms. Broker transport is 0.84 and 0.80 ms. The entire twentyfold difference occurs before either broker is involved.

Three measured properties constrain the explanation:

- It is *constant*: the 50th and 95th percentiles of TTI differ by under 0.05 ms within a run, so every event pays it. Not a tail effect.
- It is *concurrency-invariant*: 103.0, 103.0, 102.8, 102.9 ms at $N = 1, 9, 10, 12$. Not contention.
- It is *rate-dependent*: replaying identical data, code and machines at ten times real speed reduces it to 0.19–1.01 ms, and TTI to 1.8–7.2 ms.

The explanation most consistent with all three is that the client's background sender thread must wake from idle to dispatch a record and pays a fixed cost when its queue has been empty — which at 0.415 events per second is the case before essentially every event. We flag that this is inferred from the three properties rather than from instrumentation of the client's internals; a controlled comparison against a second client library is in progress and we will not assert a mechanism ahead of it.

The consequence does not depend on the mechanism, and it is our sharpest methodological result after the audit. **The difference between these two systems on this workload exists only at the workload's own arrival rate.** A benchmark driving them with a dense synthetic publisher — which is to say every benchmark in Section 2 — measures the regime in which the difference is absent, and reports parity by construction. Sparse, bursty, idle-dominated workloads are their own regime, and the risk they pose is the opposite of the usual one: not that the system is under-stressed, but that a realistic rate exposes a difference an accelerated rate conceals.

7.3 The condition that reverses the ordering

Injecting one-way delay identically at both brokers with `tc netem` reverses the ordering completely (Table 6). Kafka tracks the injected delay almost additively. Redis collapses: 4.6 seconds at 5 ms of delay, 31.3 at 20 ms and 102.5 at 50 ms, roughly 900, 1,560 and 2,050 times the delay injected. No run was truncated, so this is amplification, not loss. The Redis arm passes the consistency check on all 15 runs at every delay, for the reason given in Section 6.5.

Table 6. Injected one-way broker delay, applied identically to both systems, five feeds. Medians. The zero-delay row is rejected in both arms and marks the absence of a usable baseline.

Delay	Kafka TTI	Kafka transp.	Redis TTI	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

Magnitudes of this size cannot be a per-event cost: 102 seconds cannot be assembled from 50 ms applied once per event. The account we test is a round-trip-bound consumption pattern. Our Redis consumer acknowledges each entry with `XACK` and waits for the reply — the idiomatic pattern in client tutorials — so a read returning two hundred entries is followed by two hundred sequential round trips. On a fast network this is free: a round trip costs 0.22 ms, a ceiling of ≈4,200 exchanges per second against a demand below one. At 20 ms it has no headroom. The queueing statement is elementary [10]: once service rate falls below arrival rate the queue grows without bound and delay is set by the backlog rather than the network. Kafka's consumer serves from a prefetched buffer and commits on a timer, so it has no per-record round trip to expose.

We stated the falsification criterion in advance: if amortising the acknowledgements does not help, the account is wrong. At one feed under true real-time replay, batching acknowledgements in groups of two hundred takes median TTI from 4,138 ms to 103 ms, a factor of **40.2**, replicated four times with a spread of 4 ms unbatched and 0.4 ms batched. Read-loop instrumentation confirms the mechanism and corrects its shape: each `XREADGROUP` returns a *full* batch, a median of 106 messages, in about 32 ms, so the consumer is not read-bound. The constraint is entirely in the acknowledgement path.

An unexplained null. At five feeds under accelerated replay the same intervention does nothing: 68,960 → 73,598 ms at 20 ms delay and 87,624 → 92,386 ms at 50 ms, both marginally worse batched. We report this prominently because the obvious explanations do not survive inspection. The treatment was applied — the campaign runs a manipulation check and the consumer logs its effective batch size. The measurement is sound — the Redis arm passes on all 15 runs in each cell. The machine is not saturated in general — Kafka in the same runs sits at 78 ms. An earlier version of this work explained the null away as a consistency failure; that explanation was false and we withdraw it. We claim the mechanism at realistic load, where we have both the intervention and the read-loop evidence, and state that its behaviour at five times that load is unexplained.

7.4 Configuration dominates product

A latency comparison that ignores configuration effort misstates the engineering choice.

Each system has exactly one client setting worth one to two orders of magnitude in delay, and in each case the introductory example in the documentation uses the slow value. For Kafka it is producer pipelining: with one outstanding request per connection, which is what a synchronous send example gives, median TTI at one feed was 1,644 ms; with `max.in.flight` at 64 it was 16 ms, a factor of 103. For Redis it is acknowledgement batching: the per-message `XACK` that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both share the property that makes them hard to catch: they are *free on a co-located testbed.* Neither matters when the broker round trip is 0.2 ms. A benchmark conducted on loopback prices

both at zero and will certify as equivalent two clients that differ by two orders of magnitude in deployment. This is the practical corollary of Section 6.5: the conditions that make a testbed convenient are the conditions that make it least informative.

The systems differ in the *shape* of their configuration surface rather than its size. Kafka's latency-relevant settings are numerous, documented together, and mostly safe once pipelining is enabled. Redis's are fewer, but the consequential one is not a setting at all — it is a property of how the application's read loop is written, which no configuration audit will surface.

8 Discussion

8.1 For benchmark authors

Report a physical-consistency audit. Any delay computed as a difference of timestamps taken in different processes admits a sign check, and the check costs nothing. We rejected 58% of our own runs with it, and every rejected run had passed every conventional check we knew to apply. Better instruments exist [27]; our point is orthogonal to instrument quality — whatever instrument is used, its output should be checked against causality and the retention rate published.

Drive the benchmark at the target arrival rate. Our attribution result exists only at 0.415 events per second and vanishes at ten times that. The risk of an unrealistically slow workload is normally thought to be under-stressing the system; here the realistic rate exposed a difference the accelerated rate concealed.

Measure at a realistic round-trip time, and treat interventions as interventions. Both configuration defects in Section 7.4 are invisible on loopback, and the one intervention we ran without a manipulation check produced a clean false null.

8.2 For practitioners

The brokers themselves are equivalent within a millisecond for any workload resembling this one, and neither degrades across its concurrency range. Selection should therefore rest on durability semantics, operational familiarity and cost — not on published latency benchmarks conducted at rates the deployment will never see.

Where a consumer sits across a network, audit the *consumption pattern* before choosing a product. A client that acknowledges per message and waits for the reply converts ordinary network delay into seconds of staleness; batching removes it. Kafka is not immune by virtue of being Kafka, but because its client already amortises.

8.3 Limitations

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster arm, the durability quantification and all of Testbed A. We cannot characterise behaviour at the concurrency a multi-tenant platform would see.

The 103 *ms offset is attributed, not proven.* It is constant, concurrency-invariant and rate-dependent, which rules out contention and tail effects, but we did not instrument the client internals. If it is a property of one library rather than of Kafka, our end-to-end result is a statement about that library. It reproduced across both testbeds, four concurrency levels and 164 runs, so it is not an accident of one machine; it may be an accident of one library, and a controlled client comparison is underway.

The five-feed acknowledgement null is unexplained, as set out in Section 7.3.

One workload. The arrival-rate result should generalise to any sparse bursty feed, but we demonstrate it on one.

9 Conclusion

We set out to compare two message brokers for a real-time sports feed under varying concurrency. We obtained a complete answer, and it was physically impossible.

The audit that found it is this paper's principal contribution. A negative broker delay is proof of instrument failure, and checking for it rejected 1,321 of 2,266 runs, including every run behind a large, significant, theory-confirming result. The rejected corpus had passed plausible medians, narrow intervals, large effect sizes, the architecturally expected direction, and six prior rounds of correction. It failed only against arithmetic. The property that makes the check general is that, being a test against zero, it bites hardest exactly where the measured effect is smallest — which is to say on the delicate comparisons that benchmark papers exist to make.

Inside the check, three results survive, and none is the one we set out to find. The brokers are equivalent within a millisecond and neither degrades with concurrency, robustly to the audit's own selection effect. The end-to-end difference between them lies in client code and exists only at the workload's real arrival rate, vanishing under the accelerated replay that standard benchmarks use. And each system has one client setting worth one to two orders of magnitude, both free on the co-located testbeds where such settings are normally evaluated.

The original question turned out to be the least interesting thing we could have asked. That is worth stating plainly, because the path from it to the results above ran entirely through measurements we had every reason to trust.

Artefact Availability

All code, replay plans, aggregated results and analysis are available at https://github.com/Gustolandia/streaming-latency-sports. Event data is the StatsBomb open dataset under CC BY-NC 4.0; raw files are not redistributed but are re-fetchable from a pinned upstream commit. Every run records its git commit and per-file SHA-256. A regression suite recomputes every figure in this paper from committed data and fails if paper and data disagree.

References

[1] Arvind Arasu, Mitch Cherniack, Eduardo Galvez, David Maier, Anurag S. Maskey, Esther Ryvkina, Michael Stonebraker, and Richard Tibbetts. 2004. Linear Road: A Stream Data Management Benchmark. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. 480–491.

[2] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems* 31, 3 (2013), 1–22.

[3] ThreadSafe Diaries. 2025. I Benchmarked Kafka, RabbitMQ, and Redis Streams, The Winner Surprised Me. Accessed: June 15, 2026. https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2

[4] Bradley Efron. 1979. Bootstrap methods: another look at the jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26.

[5] Guenter Hesse, Christoph Matthies, Michael Perscheid, Matthias Uflacker, and Hasso Plattner. 2021. ESPBench: The Enterprise Stream Processing Benchmark. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. 201–212.

[6] J. L. Hodges and E. L. Lehmann. 1963. Estimates of location based on rank tests. *The Annals of Mathematical Statistics* 34, 2 (1963), 598–611.

[7] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[8] Raj Jain. 1991. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley.

[9] JusDB. 2025. Redis Streams vs Kafka: Event Streaming Architecture Comparison 2025. Accessed: June 15, 2026. https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025

[10] Leonard Kleinrock. 1975. *Queueing Systems: Volume I - Theory*. Wiley-Interscience.

[11] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation* (2011).
[12] William H. Kruskal and W. Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *J. Amer. Statist. Assoc.* 47, 260 (1952), 583–621.
[13] Daniël Lakens. 2017. Equivalence tests: a practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8, 4 (2017), 355–362.
[14] Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM* 21, 7 (1978), 558–565.
[15] Hongyou Liu, Will Hopkins, A. Miguel Gomez, and J. Sampaio Molinuevo. 2013. Inter-operator reliability of live football match statistics from OPTA Sportsdata. *International Journal of Performance Analysis in Sport* 13, 3 (2013), 803–821.
[16] Henry B. Mann and Donald R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.
[17] David L. Mills. 1991. Internet time synchronization: the network time protocol. *IEEE Transactions on Communications* 39, 10 (1991), 1482–1493.
[18] Muzeeb Mohammad. 2025. Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint arXiv:2512.16146* (2025).
[19] OpenMessaging Project, Linux Foundation. 2018. The OpenMessaging Benchmark Framework. Open standard, multi-platform messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.
[20] Luca Pappalardo, Paolo Cintia, Alessio Rossi, Emanuele Massucco, Paolo Ferragina, Dino Pedreschi, and Fosca Giannotti. 2019. A public data set of spatio-temporal match events in soccer competitions. *Scientific Data* 6, 1 (2019), 236.
[21] Salvatore Sanfilippo. 2017. Redis streams: a new data type for real-time streaming. *Redis Labs Blog* (2017). https://redis.io/topics/streams-intro
[22] Donald J. Schuirmann. 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15, 6 (1987), 657–680.
[23] Anshu Shukla, Shilpa Chaturvedi, and Yogesh Simmhan. 2017. RIoTBench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29, 21 (2017), e4257.
[24] StatsBomb. 2023. StatsBomb Open Data. *GitHub Repository* (2023). https://github.com/statsbomb/open-data
[25] Michael Stonebraker, Ugur Cetintemel, and Stan Zdonik. 2005. The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34, 4 (2005), 42–47.
[26] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. 2008. *NEXMark: A Benchmark for Queries over Data Streams.* Technical Report. OGI School of Science and Engineering, Oregon Health and Science University.
[27] Shinhyung Yang, Jiun Jeong, Bernhard Scholz, and Bernd Burgstaller. 2022. Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads. *arXiv preprint arXiv:2205.09325* (2022).
[28] Praneeth Yerrapragada. 2025. kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. Includes white paper with methodology; Accessed: June 15, 2026. https://github.com/praneethys/kafka-bullmq-benchmark